Systematization of Knowledge: Formal Verification of Consensus Protocols

Nikita Bondarev[1,2⋆], Kirill Ziborov[3,2], and Yury Yanovich[1]

[1] Skolkovo Institute of Science and Technology, Moscow, Russia
[2] Lomonosov Moscow State University, Moscow, Russia
[3] Positive Technologies, Moscow, Russia

Abstract. Formal verification is increasingly critical for blockchain consensus protocols, where subtle bugs can cause irreversible financial loss and network failure. Yet the literature on verification methods is fragmented across tools, protocol families, and property classes, hindering cumulative progress. This Systematization of Knowledge paper analyzes over 20 verified consensus protocols–from crash-fault-tolerant Raft to Byzantine-fault-tolerant HotStuff, DAG-based FairDAG, and proof-of-stake Beacon Chain–to establish a unified taxonomy of verification approaches. We introduce a verification maturity scale ranging from informal reasoning to machine-checked code proofs, and present a Protocol–Property–Method matrix mapping protocols to verified safety, liveness, and economic properties. Our analysis reveals persistent gaps: liveness verification remains underdeveloped despite its importance for progress guarantees; specification-implementation disconnects undermine real-world assurance; and scalability limits restrict verification to small networks. We provide practical recommendations for tool selection and proof engineering, and outline a research roadmap toward scalable, economically-aware verification. This work aims to guide both researchers and practitioners in building more rigorously verified consensus systems.

Keywords: Blockchain · Consensus Protocol · Formal Verification · Byzantine Fault Tolerance · Model Checking · Theorem Proving

1 Introduction

Formal verification has become an indispensable discipline in the development of blockchain systems, where even minor implementation errors can lead to irreversible financial losses, network forks, and the erosion of user trust. As consensus protocols lie at the heart of every blockchain, ensuring their correctness under adversarial conditions is paramount. This Systematization of Knowledge (SoK) paper provides a comprehensive taxonomy of formal verification approaches applied to consensus protocols, analyzing both methodological foundations and practical outcomes across diverse protocol families. The necessity of rigorous verification is underscored by the high cost of failures in decentralized environments, where updates are difficult to revert and security guarantees must be mathematically sound rather than empirically tested.

⋆ Corresponding author.

Real-world incidents underscore the need for formal verification beyond testing: arithmetic overflows in the Beacon Chain [10], liveness violations in Tendermint Fastsync [7], and unfair reward distribution in federated learning consensus [26] all escaped conventional analysis. Yet existing surveys lack a unified taxonomy connecting protocol classes, verified properties, and verification methods [40,4,36]. This work addresses that gap by establishing a verification maturity scale and a Protocol–Property–Method matrix.

We analyze diverse consensus families: crash-fault-tolerant (Raft [41]), Byzantine-fault-tolerant (Tendermint [30], HotStuff [23]), proof-of-stake (Algorand [3], Beacon Chain [2]), and DAG-based protocols (FairDAG [20], AleoBFT [13]). Each presents distinct verification challenges addressed in our taxonomy.

This SoK makes several key contributions to consensus verification:

1. A verification maturity scale from informal reasoning to machine-checked code proofs, enabling assessment of verification depth.
2. A Protocol–Property–Method matrix mapping 20+ consensus protocols to verified safety, liveness, and fairness properties and their tools.
3. Identification of critical gaps: liveness under asynchrony, dynamic stake handling, and the spec–implementation disconnect.
4. Practical recommendations for tool selection and proof engineering, grounded in protocol characteristics and verification goals.

These contributions systematize the state of the art to guide researchers and practitioners toward scalable, rigorous consensus verification.

2 Formal Properties of Consensus

Formal verification of consensus protocols is a mathematical proof that a system satisfies a certain set of properties even in the presence of failures or malicious participants. These properties are traditionally divided into safety, which ensures that nothing bad happens, and liveness, which ensures that something good eventually happens. However, modern blockchain protocols require additional guarantees beyond these classical properties, including fairness, economic incentives, and dynamic stake handling. In this section, we systematize the property classes that appear in the verification literature and explain their significance for different protocol families.

2.1 Safety Properties

Safety properties form the foundation of consensus protocol verification. Formally, a safety property is an invariant that must hold at every step of the protocol's execution; its violation implies the occurrence of an invalid state. Under the Byzantine model, where nodes may behave arbitrarily, guaranteeing safety properties even in the presence of malicious participants is a minimal requirement for establishing trust in the system. From a practical perspective, safety

properties are also more amenable to automated verification. Thus, formal verification of safety properties serves as a necessary first step toward building reliable consensus protocols. In this section, we systematize safety properties that are prioritized for verification in most papers.

At a higher level of abstraction, most safety properties verified in the literature can be unified as *consistency safety*: honest participants must not produce incompatible protocol outcomes. In single-shot consensus, this appears as *agreement* [35] on a single decided value, whereas in blockchain and state-machine-replication protocols it is typically formulated as prefix compatibility [34,18], no-fork safety, or consistency of committed logs and finalized chains. For instance, in the Algorand protocol, the asynchronous safety property is proved, demonstrating that no two distinct blocks can be certified in the same round even with full adversary control over message delivery [3].

Another safety-type property is *validity*. It requires that the decided value originate from the protocol execution itself, rather than be fabricated or injected by a malicious participant. In the formal analysis of Tendermint, this appears as the impossibility of forcing honest nodes to accept an invalid block [30]. In Red Belly Blockchain, a form of validity ensures that delivered values were broadcast by correct processes [37].

For blockchain systems, classical safety properties are often extended with specific guarantees related to data structures and finalization. No-forking guarantees that the blockchain maintains a single canonical chain where conflicting branches cannot simultaneously reach finalization. In AleoBFT, the blockchain non-forking property is proved accounting for dynamic stake changes where validators can join and leave the network every block [13]. For Raft replicated state-machine protocol safety is stated in a stronger, implementation-facing form: if one correct replica has applied a command at a log index, no other correct replica may apply a different command at that same index. Woos et al. [41] lift that execution-level consistency to linearizability, meaning that client operations appear to take effect atomically in a single global order consistent with real-time completion. For Giskard, it is proved that blocks at each stage of the protocol cannot conflict, ensuring injectivity [24].

In Proof-of-Stake protocols, *justification* and *finalization* are central safety notions used to express block irreversibility at the checkpoint level. A checkpoint is justified once it is endorsed by a supermajority of validators, while finalization additionally requires a justified descendant checkpoint, thereby turning local supermajority support into a stronger guarantee that the checkpoint belongs to the canonical history. In the Beacon Chain, these mechanisms are formally specified as the core of epoch-level finality [2]. An even stronger notion, used in Gasper [8], is *accountable safety*: if two conflicting checkpoints are ever finalized, then at least a threshold fraction of the total stake must have violated the protocol's slashing conditions [32]. Thus, unlike ordinary safety, accountable safety not only rules out conflicting finalizations under the standard fault bound, but also ensures that any such violation leaves cryptographic evidence of slashable misbehavior.

2.2 Liveness Properties

Unlike safety properties, which guarantee the absence of invalid states, liveness properties capture system progress, i.e., the requirement that "something good eventually happens". In the context of consensus protocols, this typically implies a guarantee that correct nodes will eventually reach a decision, known as *termination*, that the protocol continues extending the ledger despite failures and adverse scheduling, referred to as *progress*. In the surveyed literature, liveness verification is substantially less common than safety verification. The main reason is that liveness depends not only on the protocol logic itself, but also on temporal assumptions about timeouts, message delivery, scheduler fairness, and, in some cases, randomization. As a result, liveness proofs are typically more sensitive to the underlying execution model and are significantly harder to mechanize.

The liveness properties verified in the literature take several different forms depending on the protocol model and the proof objective. A standard formulation is eventual-progress liveness, which states that once the network becomes sufficiently well behaved, the protocol eventually makes progress and produces new decisions. This notion appears most clearly in the deductive verification of the Stellar Consensus Protocol by Losa and Dodds, who prove safety and liveness under eventual synchrony [27]. A complementary result was obtained earlier by Yoo et al., who used model checking with UPPAAL to check both safety and liveness of SCP in a timed automata model, introducing explicit timer abstractions to keep timeout-sensitive verification tractable [42]. Another notion appearing in the literature is *plausible liveness*, verified for Gasper Finality Gadget. This is more specialized property: rather than requiring that all fair executions eventually finalize, it states that from any reachable state there still exists a continuation in which checkpoints can again be finalized without violating slashing conditions, provided that more than two thirds of the stake follows the protocol. Accordingly, plausible liveness is better understood as a recoverability property for the finality gadget than as a full liveness theorem for the entire consensus stack.

Liveness proofs often require synchrony assumptions because deterministic consensus is impossible in a fully asynchronous network according to the classical Fisher-Lynch-Peterson result [16]. Therefore, proofs often implemented on partial synchrony or specific adversary models. Such as the round-rigid adversary assumption used in verifying Red Belly Blockchain to guarantee fair scheduling of transitions and termination [37]. Randomization introduces an additional difficulty, because asynchronous protocols like HoneyBadgerBFT [31] rely on random coins or probabilistic scheduling arguments to guarantee progress, but such mechanisms are difficult to encode in standard model checkers and are often unsupported directly. Additionally, the modelling complexity of modern protocols complicates liveness proofs. Tree-based models used in protocols like HotStuff make proving liveness more difficult, as ensuring tree structure and reachability predicates in inductive invariants increases verification complexity. As a result, many works focus solely on safety [23,19].

2.3 Protocol-Specific Properties

Modern consensus protocols often require formal verification of properties that go beyond classical safety and liveness. These properties arise from blockchain-specific concerns such as transaction ordering, economic attacks, broadcast semantics, auditability, and the internal structure of protocol state spaces. Literature analysis reveals several key classes of such characteristics. They become objects of formal verification to ensure the reliability of blockchain systems.

One important class is *transaction-ordering fairness*. In FairDAG, Kang et al. [20] formalize Ordering Linearizability, which requires that if all correct nodes receive transaction t_1 before t_2, then t_1 will be ordered before t_2. They also verify γ-*batch-order-fairness* for batch ordering. This property guarantees that if a fraction γ of correct nodes receives t_1 before t_2, then t_1 will not be ordered later than t_2. These properties are specific to multi-proposer blockchain settings and are intended to limit manipulation of transaction ordering. A related but distinct property appears in Fino [28], which introduces *blind order fairness*: validators must not learn transaction contents before those transactions are irrevocably committed to a total order. Unlike ordinary fairness, this property targets resistance to MEV-style attacks, especially front-running based on transaction visibility at the consensus layer.

A second class concerns broadcast-level correctness properties. In the Red Belly protocol described in [37], binary value broadcast properties are verified. These include the obligation property guaranteeing delivery if a sufficient number of nodes broadcast a value and uniformity guaranteeing that all correct nodes will deliver the value. These are protocol-specific guarantees for the correctness of the broadcast primitive on which consensus is built, rather than properties of the final consensus outcome itself. Closely related property is *delivery integrity*, emphasized in recent work on DAG-based consensus [5]. This property requires that a correct process neither delivers the same message twice nor delivers a message that was never correctly broadcast and rules out duplicates and phantom delivery.

Another protocol-specific direction concerns attack resilience and adversarial influence. In the formal analysis of Tendermint by Maung et al. [30], the authors identify a censorship-related vulnerability: the protocol may fail to include valid transactions when a sufficiently large fraction of validators strategically refuses to vote. In a related direction, Esposito et al. [14] analyze coalition attacks against Algorand, showing that coordinated adversarial behavior can bias the protocol toward committing empty blocks once the adversarial coalition exceeds the admissible threshold.

Some verified properties are better viewed as auxiliary meta-properties rather than as safety, liveness, or attack-resistance guarantees. In the Coq formalization of CBC Casper, Li et al. prove *non-triviality* and strong non-triviality [25]. These results do not state that the protocol is safe or live; rather, they show that the protocol state space is sufficiently rich to admit genuinely branching behaviors, so that safety is not obtained vacuously from a degenerate model with no meaningful alternatives. Strong non-triviality strengthens this idea by

giving a constructive witness to the branching structure of future executions. In the Agda verification of Chained HotStuff / LibraBFT, Carr et al. prove *external verifiability*, meaning that an observer who does not participate in the protocol can still check the correctness of a committed result [9].

Overall, the spectrum of formally verified properties in consensus protocols is evolving. In addition to safety and liveness, recent work verifies fairness of transaction ordering, privacy-preserving ordering guarantees, correctness of broadcast primitives, resistance to censorship and coalition attacks, and external auditability of committed outcomes. This shift reflects a broader change in the goals of formal verification: modern blockchain protocols must be shown not only to decide consistently and eventually make progress, but also to satisfy economic, structural, and adversarially robust guarantees that are specific to their intended deployment setting.

3 Verification Methods and Abstraction Levels

In this section, we analyze verification approaches for consensus protocols along two dimensions: the methodological foundation (deductive, model checking, hybrid) and the level of abstraction at which verification is performed. This distinction matters because the choice of method and abstraction jointly determines which protocol classes can be analyzed, which properties can be established, and the trade-off between automation, scalability, and proof strength (Table 1).

3.1 Interactive Theorem Proving

Deductive verification has been the most rigorous approach to work in machine-checked reasoning about consensus protocols. It typically requires translating the original algorithm, or its program implementation, into a formal model expressible in the target proof environment. Proofs are then carried out in proof assistants or solver-backed verification systems such as Rocq (formerly known as Coq), Isabelle/HOL, Agda, TLAPS, Ivy, or SMT-based frameworks, rather than by explicit state-space exploration. In this literature, the dominant proof style is inductive: authors define protocol invariants, refinement relations, or logical proof obligations and then show that every protocol step preserves them. The main advantage of this approach is that the resulting guarantees are usually parametric in the number of nodes and are not tied to a finite exploration bound, although this comes at a substantial proof-engineering cost.

A representative early result is the verification of Raft in Rocq within the Verdi framework by Woos et al [41]. The authors provide the first machine-checked proof of state machine safety for Raft and connect it to an end-to-end guarantee of linearizable state-machine replication for the extracted implementation. The proof proceeds by reasoning about the evolution of the global protocol state using a large collection of inductive invariants. However, the verified properties remain fundamentally safety-oriented, and the work does not provide a liveness proof. A closely related effort is Velisarios by Rahli et al. [35], who also

Table 1: Classification of Verified Consensus Protocols by Category, Properties, Tools & Modeling Approach

Class	Algorithm	Verified Properties	Tools & Modeling Approach	Ref.
CFT	Raft	Safety	Rocq (Verdi)	[41]
	Paxos variants	Safety, Liveness	SPIN, Isabelle/HOL · Heard-Of Model	[38,11]
	Ceph	Safety	TLA+/TLC	[15]
BFT	Ben-Or	Safety, Liveness	ByMC · Threshold automata, Round-Rigid Adversaries	[6]
	Tendermint	Safety, Liveness, Attack Resistance	PAT · CSP# Process algebra	[30]
		Safety, Liveness	TLA+/TLC, Apalache · Explicit & symbolic MC	[7]
	HotStuff	Safety	Ivy, TLA+/TLAPS · FO model	[19]
		Safety	TLA+/TLC	[23]
	Chained HotStuff/LibraBFT	Safety, External Verifiability	Agda	[9]
	TetraBFT	Safety	TLA+/Apalache · Inductive invariants	[43]
	PBFT	Safety	Rocq (Velisarios)	[35]
	Giskard	Safety	Rocq · Stage-based invariants	[24]
	Streamlet	Safety	Agda · Executable spec	[18]
	Pipelined Moonshot	Safety	Ivy	[34]
	Red Belly	Safety, Liveness, BV-Broadcast	ByMC · Parametric threshold automata	[37]
	ChonkyBFT	Safety, Liveness	Quint, Apalache · Symbolic BMC	[17]
PoS	Casper	Safety, Liveness (FFG)	Rocq · Finality gadget model	[32]
		Safety, Non-triviality (CBC)	Rocq · Layered abstraction	[25]
	Beacon Chain	Safety	PAT · CSP#, Epoch model	[2]
		Safety	Dafny · Code-level deductive verif.	[10]
	Algorand	Safety	Rocq · Timing & delay model	[3]
		Noninterference, Attack Influence	CADP/LNT · Noninterference analysis	[14]
DAG	DAG-Rider	Safety, Delivery Integrity	TLA+/TLAPS · DAG construction abstraction	[5]
	Hashgraph	Safety, Delivery Integrity	TLA+/TLAPS · DAG construction abstraction	[5]
	Aleph	Safety, Delivery Integrity	TLA+/TLAPS · DAG construction abstraction	[5]
	BullShark	Safety, Delivery Integrity	TLA+/TLAPS · DAG construction abstraction	[5]
	AleoBFT	Safety, Dynamic Stake	ACL2 · Dynamic stake model	[13]
Other	Stellar SCP	Safety, Liveness	UPPAAL · Timed automata	[42]
		Safety, Liveness	Ivy, Isabelle/HOL · FO model + interactive proof of model soundness	[27]
	STBC	Safety, Trust Model	PAT · CSP# Crowdsourcing model	[1]

use Rocq to verify PBFT. In contrast to Verdi, Velisarios is explicitly designed to model Byzantine malicious behavior, which makes it more directly applicable to BFT consensus protocols.

Another line of work formally verifies the Casper family in Rocq. Li et al. [25] formalize Correct-by-Construction Casper [8] and prove safety together with non-triviality, showing how deductive methods can clarify the mathematical assumptions underlying the protocol. A subsequent effort by Runtime Verification [32] proves accountable safety and plausible liveness for Casper FFG. However, this liveness result should be interpreted carefully: it concerns the finality gadget, not consensus in the broader sense of a complete block production and fork-choice protocol.

Deductive verification has also been applied to several other blockchain consensus protocols. Li et al. [24] specified Giskard in Rocq and formally verified its safety. For Algorand, Alturki et al. [3] develop a Rocq model that explicitly captures timing, network delay, and adversarial control of message delivery, and prove safety. For the Stellar Consensus Protocol, Losa and Dodds [27] use Ivy to verify safety and some liveness properties for arbitrary but fixed quorum configurations in first order logic together with Isabelle/HOL used to prove the soundness of the first-order Ivy model.

For the HotStuff family, the deductive literature develops in two main directions. Jehl [19] verifies a simplified version of HotStuff using Ivy and the TLA Proof System (TLAPS). The contribution is explicitly a proof of safety, and one of its main observations is methodological: HotStuff's tree-shaped structure makes verification more difficult than the more traditional view-instance model used in earlier consensus protocols. A different approach is taken by Carr et al. [9], who formalize the Chained HotStuff/LibraBFT core protocol in Agda. Their proof establishes the protocol's safety property and an additional condition enabling external verifiability of committed results by non-participating observers. They explicitly leave liveness outside the proved model, and their formalization covers only a single epoch or configuration, leaving reconfiguration for future work. A more recent solver-aided example is Praveen et al. [34] on Pipelined Moonshot, a HotStuff-style BFT protocol. Using Ivy, they prove safety, showing that as long as fewer than one third of validators are Byzantine, the protocol does not admit forks.

A similar proof-assistant style appears in Jaskelioff et al. [18], who present an Agda formalization of the Streamlet consensus protocol. Their contribution is a mechanized proof of consistency, with particular emphasis on making the formalization both readable and computable, so that it can serve not only as a proof artifact but also as a basis for testing implementations.

More recently, deductive verification has started to move beyond one-off proofs of individual protocols toward reusable proof frameworks. In Bertrand et al. [5], the authors develop TLA+ specifications with TLAPS proofs for a family of DAG-based consensus protocols, including DAG-Rider, Cordial Miners, Hashgraph, Eventual Synchronous BullShark, and a variation of Aleph. The main verified property is safety. The proof is organized around reusable compo-

nents for DAG construction and ordering, which reduces repeated proof effort across related protocols. This suggests that deductive verification may scale not only by proving isolated protocols correct, but also by developing reusable libraries of proof components.

Overall, the deductive literature is dominated by proofs of safety-style properties. Verified liveness is substantially less common, with Stellar and Casper FFG being notable exceptions; even there, the liveness result either depends on explicit synchrony assumptions or applies to a narrower finalization component rather than to the full consensus stack. We are not aware of deductive-only works in this set whose primary verified contribution is fairness or censorship resistance.

The strengths of deductive verification are clear: it provides the strongest form of machine-checked assurance and avoids the bounded-state limitations of model checking. In addition, code extraction from proof assistants into executable languages can narrow the gap between a verified formalization and a runnable artifact, and in some cases enables comparison with implementation-level code. At the same time, the approach has significant drawbacks. First, it is highly labor-intensive, often requiring many auxiliary invariants and substantial proof engineering. Second, it scales poorly to liveness, since liveness proofs must encode subtle assumptions about time, fairness, scheduling, and partial synchrony. Finally, such proofs are often fragile under protocol evolution: relatively small changes in the protocol logic may require extensive proof revision.

3.2 Code-Level Verification

Not all deductive work in the corpus operates at the level of a clean protocol theorem. Cassez et al. [10] verify a large, critical fragment of the Ethereum 2.0 Beacon Chain reference implementation in Dafny. Their main result is the verified absence of runtime errors together with additional functional-correctness specifications strong enough to uncover bugs and justify verified fixes. This is still deductive verification, but the verified properties are closer to software correctness and implementation safety than to consensus-level theorems.

3.3 Model Checking

Model checking is an automated verification technique that involves the systematic exploration of the reachable states of a formal model to determine whether properties hold. Protocols are typically modeled as transition systems or finite-state abstractions, while properties are expressed in temporal logics such as Linear Temporal Logic (LTL) or Computation Tree Logic (CTL). Compared with interactive theorem proving, model checking requires much less manual proof effort and is especially effective at finding concrete counterexamples. This makes it particularly valuable in consensus verification, where subtle bugs often arise from rare interleavings of messages, faults, and timeouts. A representative example is the verification of Tendermint Fastsync, where TLC exposed a counterexample violating termination and helped identify a protocol flaw during refinement [7].

The primary limitation of model checking is the exponential growth in the number of reachable states with an increase of number of processes, rounds, messages, and timer values, which usually restricts explicit-state verification to small configurations: often with $\leq$10 processes and without explicit modelling of timing [38,42,23].

The analyzed works utilize a wide spectrum of model checking tools. Despite sharing a common goal—state space exploration—they differ in methods of state representation, supported logics, and scope of application. The choice of tool is dictated by the specifics of the protocol and scalability requirements.

Explicit State Checkers encompass tools that verify system correctness by explicitly storing and traversing every reachable state in memory to construct a reachability graph. While intuitive and effective for finding bugs in early design stages, these tools are inherently constrained by the state explosion problem. The primary representatives of this class include TLC (the model checker for TLA+), PAT (Process Analysis Toolkit), and SPIN (using the PROMELA language).

TLC is widely used for debugging distributed protocols due to its ability to generate concrete counterexamples. A notable application is found in the work of Braithwaite et al. [7] on the Tendermint Fastsync protocol. The authors developed a TLA+ multi-level specification and successfully employed TLC to identify a critical counterexample violating the termination property during the refinement phase. This demonstrated TLC's strength in catching logical errors early. However, the study also highlighted severe scalability limits: as the number of peers or the blockchain height increased, TLC rapidly exhausted computational resources. For complex liveness checks, the tool timed out after 24 hours, failing to complete an exhaustive search.

The limitations of explicit checking were further underscored by Yu et al. [43] in their verification of TetraBFT. Their initial attempt to verify safety using TLC failed even for a minimal configuration of 4 nodes (1 Byzantine), 3 proposed values, and 5 views, as the state space exploded immediately. A complementary case study is provided by Fernandes [15] in his formal verification of the Ceph consensus algorithm, a Paxos-variant for distributed storage. The authors used TLC to verify safety properties (agreement and validity) for configurations up to 4 monitors and 2 proposed values. They successfully reproduced a known bug from an early Ceph version via counterexample generation, demonstrating TLC's practical utility for regression testing. To mitigate state explosion, the authors employed symmetry reduction and bounded constants. Despite these optimizations, verification for 4 monitors required processing 206 million states and took approximately 11 minutes, with exponential growth observed beyond this point.

Similar scalability constraints appear in PAT-based verifications. PAT utilizes the CSP# language, which uniquely combines high-level parallelism operators with low-level programming constructs like variables and arrays. Afzaal et al. [1,2] leveraged PAT to formally specify and verify consensus mechanisms for crowdsourcing (STBC) and the justification/finalization logic of the Ethereum Beacon Chain. The authors successfully proved absence of forks and fault tol-

erance guarantees. Despite these successes, empirical results revealed the classic bottleneck of explicit model checking: verification time and memory consumption exhibited exponential growth as the number of nodes or epochs increased.

Comparable limits were observed with SPIN. SPIN, based on the PROMELA language, has been a staple for verifying consensus algorithms within the Heard-Of model. Tsuchiya et al. [38] applied SPIN to verify Paxos-type algorithms (LastVoting and Hybrid-1), employing an optimization that abstracted away specific Heard-Of sets by representing all possible behaviors non-deterministically. Despite such optimizations, SPIN's scalability remains fundamentally limited by the number of processes. As documented in technical reports, even with aggressive state-space reductions, SPIN could only verify agreement for up to 3 processes. In contrast, approaches utilizing Bounded Model Checking (BMC) with SMT solvers extended this limit to 8–11 processes.

Together, these case studies illustrate a consistent pattern across explicit checkers. The collective experience with TLC, PAT, and SPIN demonstrates that while explicit state checkers are powerful tools for debugging specifications and finding shallow bugs via counterexamples, they are ill-suited for proving correctness in realistic, large-scale blockchain environments. The exponential state growth restricts verification to toy configurations (often $N \leq 4$), as seen in the TetraBFT, Tendermint, and Ceph case studies. Even with engineering optimizations—symmetry reduction, bounded constants, or custom visualizers—explicit enumeration cannot overcome the fundamental combinatorial barrier.

This limitation has driven a methodological shift in contemporary research: explicit checkers are increasingly used as a first step in a hybrid verification workflow, complemented by symbolic model checking or inductive invariant proofs to achieve scalable, parameterized guarantees.

Symbolic and SMT-based Checkers encompass tools that represent sets of states symbolically via logical formulas, delegating state exploration to SMT solvers (e.g., Z3, cvc5). This approach covers significantly larger state spaces than explicit enumeration. Prominent tools include Apalache [21] for TLA+ and ByMC [22] for parameterized threshold-guarded algorithms.

Bounded Model Checking (BMC) with SMT solvers has been pivotal for verifying consensus algorithms with large or infinite state spaces. Tsuchiya et al. [38] reduced asynchronous consensus verification to single-phase problems solvable by SMT, scaling to ∼10 processes versus 3–4 for explicit checkers. Braithwaite et al. [7] showed Apalache finding Tendermint Fastsync errors in 10 minutes where TLC failed to terminate.

For parameterized correctness, ByMC leverages cutoff techniques and SMT solving. Tholoniat and Gramoli verified Red Belly Blockchain safety and liveness for arbitrary $N \geq 3F + 1$ [37].

Symbolic checkers handle larger parameter spaces with high automation, but face key constraints. BMC requires bounds on nodes/rounds. SMT solvers lack native support for probabilistic liveness, leaving randomized protocol termination unverified. Specifications outside decidable first-order fragments require manual abstraction. Symbolic and SMT-based checkers significantly advance

over explicit methods, enabling verification of deeper executions and larger configurations. However, bounded checks do not constitute full proofs, and probabilistic liveness remains out of reach. Modern research thus more often combines symbolic BMC with inductive invariant proofs or parameterized verification for comprehensive guarantees.

Beyond general-purpose checkers, specialized tools address domain-specific challenges. UPPAAL models timed automata to verify timeout-critical protocols like Stellar SCP, explicitly capturing asynchronous networks and synchronization for safety and liveness analysis [42]. ByMC enables parameterized verification via threshold automata, abstracting pseudo-code into message-count guards to prove properties for arbitrary $n \geq 3f + 1$ in round-based Byzantine protocols. While these tools trade generality for depth—enabling rigorous analysis of timing and scalability that generic checkers cannot efficiently handle—their applicability remains constrained by specific protocol structures.

3.4 Hybrid Approaches and Emerging Frameworks

Hybrid verification has emerged as a natural direction for the formal analysis of consensus protocols, combining the automation of solver-based reasoning with the expressiveness of interactive proof. Such an approach makes it possible to specify protocols in a transition-system style amenable to automation, while retaining the ability to discharge difficult proof obligations manually when automation becomes insufficient. A recent example is Veil by Pîrlea et al. [33], a framework embedded in Lean for the automated and interactive verification of transition systems. Veil allows the user to describe a transition system and its specification in a simple imperative language, supports bounded model checking through Lean tactics, and generates verification conditions in first-order logic that are discharged automatically by SMT solvers. Its evaluation includes consensus-related case studies such as Paxos and Stellar, the latter verified within a single framework rather than through the cross-tool combination of Ivy and Isabelle/HOL used by Losa and Dodds [27]. This illustrates the appeal of hybrid methods for consensus verification: they can reduce the trusted gap between modeling and proof while preserving a significant degree of automation. At the same time, as of March 2026, Veil framework does not yet support liveness verification natively.

4 Abstraction Techniques for Scalability

In this section, we discuss abstraction techniques used in the formal verification of consensus protocols. In practice, verification rarely targets the entire production implementation; instead, it is typically carried out over an abstract model of the protocol's core logic or over a simplified protocol variant. This is particularly visible in the HotStuff literature: deductive proofs reason about an abstract single-epoch core independent of implementation [19,9], while model-checking work targets simplified TLA+ specifications rather than production code [23].

More broadly, most verification efforts omit cryptographic primitives or model them axiomatically, and treat network semantics separately from the protocol transition system itself. As a result, regardless of the verification methodology, one needs a sound operational abstraction that isolates the protocol logic from implementation details while preserving the properties of interest.

Approach-specific abstractions then build on top of this common operational layer. Deductive verification generally tolerates a lower level of abstraction than model checking, thanks to the richer semantics of proof assistants such as Rocq, Isabelle/HOL, and Agda. A particularly important technique is property-oriented abstraction hierarchy, where different properties are proved at different abstraction levels. The clearest example is CBC Casper, whose Coq development explicitly introduces several abstraction layers and uses them to separate the assumptions needed for safety from those needed for non-triviality [25].

By contrast, the primary limitation of classical model checking is the exponential growth of the state space as the number of processes, message histories, or timer values increases. Time is a particularly acute source of blow-up. In consensus protocols, explicit modeling of timeouts can easily generate many states that differ only in clock valuations while contributing little to safety reasoning. For this reason, safety analyses often abstract timeouts into nondeterministic events, whereas liveness arguments usually require a more faithful treatment of the relationship between timeout expiration, message delay, and processing time. This tension is visible in work on Tendermint and SCP. Tendermint Fast-sync specifications [7] make clear that the consensus relies critically on timeout progression for liveness, which makes timer abstraction delicate; similarly, verification of SCP showed that timer- and delay-aware modeling is necessary in order to expose liveness-sensitive behaviors that disappear in purely untimed abstractions. Yoo et al. [42] therefore introduced explicit timing abstractions by parameterizing the maximum timer value and the gap between node clocks, allowing UPPAAL to check safety and liveness properties of SCP in a finite time while preserving timeout-dependent logic under different quorum configurations.

A different response to the scalability limits of model checking is to move from concrete process interleavings to parameterized symbolic models. The most influential example in the consensus literature is the use of threshold automata and the Byzantine Model Checker (ByMC) [22]. **Threshold automata (TA)** provide an abstraction for fault-tolerant distributed algorithms in which the global state is represented by counters over local control locations and transitions are guarded by threshold conditions over shared counters or parameters. This representation is especially well suited to consensus protocols whose control flow is driven by quorum predicates such as receiving at least $N - F$ messages, since these guards can be encoded directly without enumerating individual process identities. As a result, ByMC supports automatic parameterized verification for arbitrary number of processes and admissible fault thresholds, rather than for a few fixed instances. This abstraction has been used both for asynchronous randomized consensus, including Ben-Or's and Bracha's algorithms under round-rigid adversaries [6], and for blockchain-style Byzantine consensus, most notably

in the verification of Red Belly, where the protocol is decomposed into threshold-automaton models of inner broadcast and outer decision logic [37]. The main limitation of the approach is that its scalability relies on strong structural regularity: TA are most natural for symmetric, threshold-driven, round-based protocols, and become less natural for consensus designs with dynamic committees or highly asymmetric control flow. In addition, applying the technique in practice typically requires non-trivial manual modeling effort and substantial expertise in constructing sound and faithful threshold-automaton abstractions of the original protocol.

4.1 Communication Model Abstractions

Beyond threshold abstractions, another major family of communication-model abstractions is the **Heard-Of (HO) model** [12]. In HO, the network is abstracted by the sets $HO(p, r)$ of processes from which process p actually receives messages in round r. Timing assumptions, omission faults, and benign failures are then encoded not by explicit schedules of sends and receives, but by predicates over collections of heard-of sets. This shifts verification from low-level reasoning about message orderings to higher-level reasoning about round-based communication patterns. That abstraction proved effective for scalability: early work by Tsuchiya and Schiper [38,39] used HO to reduce consensus verification to finite or phase-bounded obligations amenable to model checking, while Charron-Bost and Merz used Isabelle/HOL to verify Paxos at the HO level with a comparatively compact proof structure [11]. More recent work pursued cutoff-style reasoning, aiming to lift small-instance verification results to unbounded system sizes [29], while HOME adapted the HO perspective to threshold-guarded BFT protocols through a dedicated modeling and verification environment [44]. At the same time, HO remains a specialized abstraction. It is most natural for fixed-process, round-based algorithms and considerably less natural for modern blockchain protocols with dynamic committees, threshold certificates, and DAG-based ordering. Thus, HO is a powerful but domain-specific abstraction: it yields scalability gains for well-structured consensus families but is less suited to contemporary blockchain protocols with dynamic committees or DAG-based ordering.

Another clear example of communication-level abstraction appears in [5], which distinguishes between reliable broadcast and unreliable communication specifications of DAG-based protocols. Authors do not model concrete network implementations; instead, they introduce abstract communication specifications chosen to preserve exactly the safety-relevant behavior needed for their proofs. For protocols based on reliable broadcast, they deliberately weaken the abstraction and retain only the integrity aspect of reliable broadcast, explicitly dropping validity and agreement because these are only needed for liveness and lie outside the scope of their safety proofs. For protocols based on unreliable communication, they abstract away implementation details even further: receiving a vertex is modeled as a process non-deterministically choosing a vertex from another pro-

cess's local DAG and copying it, thereby capturing all asynchronous interleavings that a concrete gossip- or broadcast-style implementation could produce.

4.2 State Management Abstractions

A distinct abstraction pattern in recent deductive verification is state management abstraction, where dynamic participation is modeled without introducing dynamic process creation or destruction into the formal semantics. A clear example is the ACL2 verification of AleoBFT by Coglio and McCarthy [13], which targets a DAG-based BFT protocol with dynamic stake. Instead of modeling validators as entities that appear and disappear over time, the formal model keeps a static universe of validator addresses in the global state and derives the currently relevant validator sets from blockchain state. Committees are not stored as primitive state components; they are derived from the current blockchain prefix, with validator bonding and unbonding affecting committee membership only after a fixed lookback delay. This avoids explicit reasoning about process initialization and membership churn, while still capturing highly dynamic participation. Stake is handled similarly: bonding and unbonding are recorded in blocks, and the active committee with its stake distribution is computed from the resulting blockchain state. The proof obligation shifts from reasoning about reconfiguration events to reasoning about invariants over the mapping from blockchain state to validator sets. This makes the model well suited for inductive safety proofs, while preserving the essential feature that committees may change completely from block to block, subject only to the protocol's fixed lookback rule and the assumption that each derived committee contains less than one-third faulty stake.

5 Research Challenges and Open Problems

Our analysis identifies four persistent gaps that limit the impact of formal verification on blockchain consensus.

Liveness and Economic Guarantees. Safety verification dominates the literature, with only $\approx 30\%$ of surveyed works addressing liveness. This imbalance stems from the inherent difficulty of modeling partial synchrony, timeout progression, and probabilistic scheduling—all required to bypass the FLP impossibility result. Beyond classical termination, modern protocols demand verification of transaction finality latency, censorship resistance, and MEV-resistant ordering fairness. These properties blur the line between functional correctness and economic performance, requiring frameworks that integrate distributed systems reasoning with game-theoretic models of rational adversaries. No existing tool handles dynamic stake, liveness, and economic incentives in a unified manner.

Scalability and Parameterization. State-space complexity severely restricts verification depth. Explicit checkers typically exhaust resources beyond 4–10 nodes, while symbolic BMC scales only marginally further. Parameterized

verification via threshold automata elegantly handles symmetric, round-based protocols, but struggles with modern DAG-based designs and dynamic validator sets. Protocols like AleoBFT introduce cyclic dependencies between consensus logic and stake evolution, necessitating novel abstractions (e.g., lookback windows) to decouple membership changes from round boundaries. Automating these abstractions remains an open challenge.

Specification–Implementation Gap. Most verified models target abstract TLA+ or Rocq specifications that diverge significantly from production Go/Rust code. This disconnect creates assurance blind spots: verified theorems do not guarantee implementation correctness, and protocol upgrades quickly invalidate verification artifacts. While isolated efforts like Dafny-based Beacon Chain verification narrow this gap, systematic approaches—model extraction from code, verified code generation, and runtime monitoring—remain underdeveloped and difficult to integrate into continuous deployment pipelines.

Methodological Fragmentation. Verification efforts are predominantly one-off, using bespoke models and ad-hoc tool choices. Lack of standardized benchmarks, reusable component libraries (e.g., for reliable broadcast or view synchronization), and systematic abstraction guidelines hinders cumulative progress. The field needs shared infrastructure that enables empirical comparison of tools, lowers the expertise barrier, and supports incremental assurance from early design to production deployment.

6 Recommendations and a Path Forward

We propose a three-tier roadmap to advance consensus verification from isolated proofs to integrated engineering practice.

Immediate: Pragmatic Tool Matching. Practitioners should align verification depth with risk tolerance. Explicit model checking (TLC, SPIN) suits early-stage design exploration for $n \leq 4$; symbolic BMC (Apalache, Quint) scales to ≈ 10 nodes for bounded safety; timed automata (UPPAAL) are essential for timeout-critical protocols. For unbounded guarantees or dynamic stake, deductive provers (Rocq, Agda, Isabelle) or hybrid frameworks (Veil, Ivy) are necessary despite higher proof-engineering costs. Code-level verification (Dafny) should target critical state-transition logic to bridge the spec–implementation gap.

To ensure maintainability, adopt modular proof engineering: separate communication assumptions from ordering logic, define interface lemmas for local reasoning, and invest in reusable specifications for primitives like reliable broadcast and threshold signatures. Where possible, refine abstractions incrementally—start with high-level models (e.g., Heard-Of) for scalability, then concretize for implementation verification.

Medium-Term: Standardized Infrastructure. Develop integrated pipelines combining specification-guided fuzzing, symbolic execution, and deductive proofs for incremental assurance. Prioritize benchmark suites spanning CFT, BFT, PoS, and DAG families with reference implementations, formal specs, and known vulnerabilities. Automated abstraction synthesis—using program analysis or ML

Table 2: Tool Selection Guide: Matching Protocol Characteristics to Verification Methods

Protocol Feature	**Recommended Approach**	**Tools**	**Rationale / Limitations**
Early design analysis, bug-finding	Explicit-state Model Checking	TLC, PAT, SPIN	Fast feedback on small configs ($n \leq 4$); not exhaustive for large systems
Deep simulations, inductive invariants	Symbolic MC	Apalache, Quint	Better scaling than explicit (up to $n \approx 10$); requires TLA+ or SMT encoding
Timeouts, real-time logic	Timed Automata MC	UPPAAL	Explicit time modeling; state explosion for large configs
Arbitrary n, threshold guards	Parametric MC (Threshold Automata)	ByMC	Proves for any $n \geq 3f + 1$; requires symmetric, round-based structure
Parametric safety, Complex invariants: dynamic stake, economic properties, adversarial strategies	Deductive verification in ITP or hybrid frameworks	Rocq, Isabelle/HOL, Agda, Ivy, Veil	Strongest guarantees; high proof-engineering cost, liveness hard
Implementation-level correctness	Deductive Code Verification	Dafny	Closes spec-impl gap; labor-intensive, protocol-agnostic properties

to derive threshold or HO abstractions from code—would lower the expertise barrier significantly.

Long-Term: Unified Economic-Security Frameworks. Future work must merge safety, liveness, and economic rationality into single verification frameworks. This requires extending temporal logic with probabilistic and game-theoretic semantics to reason about strategic deviations, MEV resistance, and finality latency under rational adversaries. The ultimate goal is certified end-to-end implementations: domain-specific consensus languages compiled to verified bytecode, backed by proof-carrying code and formally verified runtime systems. Modular proof engineering and reusable component libraries will be essential to scale this vision.

7 Conclusion

This SoK systematizes formal verification approaches for blockchain consensus, mapping over thirty protocols to a unified taxonomy of properties, methods, and abstractions. While significant progress has been made in verifying safety for major protocols like Raft, Tendermint, and Algorand, critical gaps persist. Liveness verification remains underdeveloped, scalability constraints limit analysis to toy configurations, and verified models frequently diverge from production code. Moreover, the lack of standardized tooling and reusable abstractions hinders cumulative progress.

Addressing these challenges requires a paradigm shift: from isolated, safety-focused proofs to scalable, economically-aware verification integrated into the

protocol development lifecycle. By adopting pragmatic tool-selection strategies, investing in modular proof engineering, and developing unified frameworks for dynamic stake and rational adversaries, the community can deliver the rigorous guarantees demanded by decentralized systems securing billions in value. Formal verification must evolve from a post-hoc audit tool to a foundational engineering practice.

References

1. Afzaal, H., Imran, M., Janjua, M.U., Gochhayat, S.P.: Formal modeling and verification of a blockchain-based crowdsourcing consensus protocol. IEEE Access **10**, 8163–8183 (2022)
2. Afzaal, H., Zafar, N.A., Tehseen, A., Kousar, S., Imran, M.: Formal verification of justification and finalization in beacon chain. IEEE Access **12**, 55077–55102 (2024)
3. Alturki, M.A., Chen, J., Luchangco, V., Moore, B., Palmskog, K., Peña, L., Roşu, G.: Towards a verified model of the algorand consensus protocol in coq. In: Formal Methods. FM 2019 International Workshops: Porto, Portugal, October 7–11, 2019, Revised Selected Papers, Part I. p. 362–367. Springer-Verlag, Berlin, Heidelberg (2019)
4. Bano, S., Sonnino, A., Al-Bassam, M., Azouvi, S., McCorry, P., Meiklejohn, S., Danezis, G.: Sok: Consensus in the age of blockchains. In: Proceedings of the 1st ACM Conference on Advances in Financial Technologies. p. 183–198. AFT '19, Association for Computing Machinery, New York, NY, USA (2019)
5. Bertrand, N., Ghorpade, P., Rubin, S., Scholz, B., Subotić, P.: Reusable formal verification of dag-based consensus protocols. In: NASA Formal Methods: 17th International Symposium, NFM 2025, Williamsburg, VA, USA, June 11–13, 2025, Proceedings. p. 138–158. Springer-Verlag, Berlin, Heidelberg (2025)
6. Bertrand, N., Konnov, I., Lazić, M., Widder, J.: Verification of randomized consensus algorithms under round-rigid adversaries. Int. J. Softw. Tools Technol. Transf. **23**(5), 797–821 (Oct 2021)
7. Braithwaite, S., Buchman, E., Konnov, I., Milosevic, Z., Stoilkovska, I., Widder, J., Zamfir, A.: Formal Specification and Model Checking of the Tendermint Blockchain Synchronization Protocol. In: Bernardo, B., Marmsoler, D. (eds.) 2nd Workshop on Formal Methods for Blockchains (FMBC 2020). Open Access Series in Informatics (OASIcs), vol. 84, pp. 10:1–10:8. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2020)
8. Buterin, V., Griffith, V.: Casper the friendly finality gadget. CoRR **abs/1710.09437** (2017)
9. Carr, H., Jenkins, C., Moir, M., Miraldo, V.C., Silva, L.: Towards formal verification of hotstuff-based byzantine fault tolerant consensus in agda. In: NASA Formal Methods: 14th International Symposium, NFM 2022, Pasadena, CA, USA, May 24–27, 2022, Proceedings. p. 616–635. Springer-Verlag, Berlin, Heidelberg (2022)
10. Cassez, F., Fuller, J., Asgaonkar, A.: Formal Verification of the Ethereum 2.0 Beacon Chain, vol. 13243, pp. 167–182. Springer (2022)
11. Charron-Bost, B., Merz, S.: Formal verification of a consensus algorithm in the heard-of model. International Journal of Software and Informatics **3**, 273–303 (2009)
12. Charron-Bost, B., Schiper, A.: The heard-of model: Computing in distributed systems with benign faults. Distributed Computing **22**(1), 49–71 (2009)

13. Coglio, A., McCarthy, E.: Formal Verification of Blockchain Nonforking in DAG-Based BFT Consensus with Dynamic Stake (2025)
14. Esposito, A., Rossi, F.P., Bernardo, M., Fabris, F., Garavel, H.: Formal modeling and verification of the algorand consensus protocol in cadp (2025)
15. Fernandes, A.d.N.: Formal Verification of the Ceph Consensus Algorithm Using TLA+. Ph.D. thesis, University of Porto (2021)
16. Fischer, M.J., Lynch, N.A., Paterson, M.S.: Impossibility of distributed consensus with one faulty process. J. ACM **32**(2), 374–382 (Apr 1985)
17. França, B., Kolegov, D., Konnov, I., Prusak, G.: Chonkybft: Consensus protocol of zksync (2025)
18. Jaskelioff, M., Melkonian, O., Chapman, J.: A Readable and Computable Formalization of the Streamlet Consensus Protocol. In: 6th International Workshop on Formal Methods for Blockchains (FMBC 2025). Open Access Series in Informatics (OASIcs), vol. 129, pp. 7:1–7:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2025)
19. Jehl, L.: Formal verification of hotstuff. In: Formal Techniques for Distributed Objects, Components, and Systems. p. 197–204. Springer-Verlag, Berlin, Heidelberg (2021)
20. Kang, D., Chen, J., Dinh, T.T.A., Sadoghi, M.: Fairdag: Consensus fairness over multi-proposer causal design (2026)
21. Konnov, I., Kukovec, J., Tran, T.H.: Tla+ model checking made symbolic. Proc. ACM Program. Lang. **3**(OOPSLA) (Oct 2019)
22. Konnov, I., Widder, J.: Bymc: Byzantine model checker. In: ISoLA 2018 Proceedings. p. 327–342. Springer-Verlag, Berlin, Heidelberg (2018)
23. Kukharenko, V., Ziborov, K., Sadykov, R., Rezin, R.: Verification of hotstuff bft consensus protocol with tla+/tlc in an industrial setting. SHS Web of Conferences **93**, 01006 (01 2021)
24. Li, E., Palmskog, K., Sebe, M., Roşu, G.: Specification of the giskard consensus protocol (2020)
25. Li, E., Serbanuta, T., Diaconescu, D., Zamfir, V., Rosu, G.: Formalizing correct-by-construction casper in coq. In: 2020 IEEE International Conference on Blockchain and Cryptocurrency (ICBC). pp. 1–3. IEEE (5 2020)
26. Liu, H., Zhu, F., Cheng, L.: Proof-of-data: A consensus protocol for collaborative intelligence (2025)
27. Losa, G., Dodds, M.: On the Formal Verification of the Stellar Consensus Protocol. In: 2nd Workshop on Formal Methods for Blockchains (FMBC 2020). Open Access Series in Informatics (OASIcs), vol. 84, pp. 9:1–9:9. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2020)
28. Malkhi, D., Szalachowski, P.: Maximal extractable value (mev) protection on a dag (2022)
29. Marić, O., Sprenger, C., Basin, D.A.: Cutoff bounds for consensus algorithms. In: CAV 2017, Proceedings, Part II. Lecture Notes in Computer Science, vol. 10427, pp. 217–237. Springer (2017)
30. Maung Maung Thin, W.Y., Dong, N., Bai, G., Dong, J.S.: Formal analysis of a proof-of-stake blockchain. In: 2018 23rd International Conference on Engineering of Complex Computer Systems (ICECCS). pp. 197–200 (2018)
31. Miller, A., Xia, Y., Croman, K., Shi, E., Song, D.: The honey badger of bft protocols. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. p. 31–42. CCS '16, Association for Computing Machinery, New York, NY, USA (2016)

32. Palmskog, K., Gligoric, M., Pena, L., Moore, B., Roşu, G.: Verification of Casper in the Coq Proof Assistant
33. Pîrlea, G., Gladshtein, V., Kinsbruner, E., Zhao, Q., Sergey, I.: Veil: A framework for automated and interactive verification of transition systems. In: CAV 2025 Proceedings. p. 26–41. Springer-Verlag, Berlin, Heidelberg (2025)
34. Praveen, M., Ramesh, R., Doidge, I.: Formally Verifying the Safety of Pipelined Moonshot Consensus Protocol. In: 5th International Workshop on Formal Methods for Blockchains (FMBC 2024). Open Access Series in Informatics (OASIcs), vol. 118, pp. 3:1–3:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2024)
35. Rahli, V., Vukotic, I., Völp, M., Esteves-Verissimo, P.: Velisarios: Byzantine fault-tolerant protocols powered by coq. In: Ahmed, A. (ed.) Programming Languages and Systems. pp. 619–650. Springer International Publishing, Cham (2018)
36. Raikwar, M., Polyanskii, N., Müller, S.: Sok: Dag-based consensus protocols. In: 2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC). pp. 1–18 (2024)
37. Tholoniat, P., Gramoli, V.: Certifying blockchain byzantine fault tolerance. CoRR **abs/1909.07453** (2019)
38. Tsuchiya, T., Schiper, A.: Using bounded model checking to verify consensus algorithms. In: 22nd International Symposium on Distributed Computing (DISC 2008). Lecture Notes in Computer Science, vol. 5218, pp. 466–480. Springer (2008)
39. Tsuchiya, T., Schiper, A.: Verification of consensus algorithms using satisfiability solving. Distributed Computing **23**(5–6), 341–358 (2010)
40. Verma, S., Yadav, D., Chandra, G.: Introduction of formal methods in blockchain consensus mechanism and its associated protocols. IEEE Access **10**, 66611–66624 (2022)
41. Woos, D., Wilcox, J.R., Anton, S., Tatlock, Z., Ernst, M.D., Anderson, T.: Planning for change in a formal verification of the raft consensus protocol. In: Proceedings of the 5th ACM SIGPLAN Conference on Certified Programs and Proofs. p. 154–165. CPP 2016, Association for Computing Machinery, New York, NY, USA (2016)
42. Yoo, J., Jung, Y., Shin, D., Bae, M., Jee, E.: Formal modeling and verification of a federated byzantine agreement algorithm for blockchain platforms. In: 2019 IEEE International Workshop on Blockchain Oriented Software Engineering (IWBOSE). pp. 11–21 (2019)
43. Yu, Q., Losa, G., Wang, X.: Tetrabft: Reducing latency of unauthenticated, responsive bft consensus (2024)
44. Zhai, S., Li, X., Ge, N.: Home: Heard-of based formal modeling and verification environment for consensus protocols. In: 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion 2023). pp. 63–67. IEEE (2023)